Learning Objectives

Learners will be able to…

- **Define the Tree ADT**
- **Define commonly used vocabulary regarding trees**
- **Implement and print a tree**

Introduction to Trees ADT

Non-Linear Data Structures

Data structures are pivotal, providing foundational means of managing and organizing data. We know that we categorize these structures into linear and nonlinear types based on their organization and data storage.

Linear data structures have their elements arranged in a sequential manner, where each element has a unique predecessor and successor except the first and last elements. Examples of linear data structures are Arrays, Linked Lists, Stacks, and Queues.

```
7 - 6 - 8 - 9 - 0
```

Contrasting with the linear models, non-linear data structures do not follow a sequential path. They are organized hierarchically, meaning elements in these structures possess parent-child relationships. Trees and graphs (introduced in a later module) are primary examples of non-linear data structures.



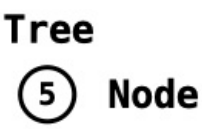
Introduction to Trees

A **tree** is a crucial non-linear data structure that emulates a tree structure with a set of linked **nodes**. The topmost node is the **root**, and the elements attached to it are its **children**, forming a hierarchy. Each child node can have its own children, known as sub-trees.

info

Trees Vocabulary

- **Root:** The top node in a tree.
- **Leaf:** A node with no children.
- **Node:** A node with at least one child.
- **Edge:** The link between two nodes.
- **Height:** The length of the longest path to a leaf.
- **Depth:** The length of the path to its root.



Trees are beneficial when it comes to representing hierarchical structures, such as the file system on a computer. They also provide efficient insertion and search operations, making them suitable for various applications like databases and computer graphics. In future assignments, we will see how the non-linear (hierarchical) structure gives trees efficiencies in operations that are just not possible with linear data structures. Trees offer flexibility and versatile representations of data relationships.

Tree Vocabulary

Tree Terminology and Properties

Before we proceed to the in-depth discussion and implementation of trees, it is crucial to understand the essential terminologies and properties associated with trees.

Terminology

Use the interactive image to the left to help you better understand the terminology surrounding trees. Mouse over the buttons to see a representation of each term:

- **Node** - A **node** is the fundamental building block of a tree, holding the data and providing the connection to other nodes.
- **Edge** - An **edge** is a link between two nodes, representing the relationship between them.
- **Root** - The **root** is the topmost node of a tree, having no parent. It is the node from where the tree originates.
- **Leaf Node** - A **leaf node** is a node with no children, located at the periphery of the tree.
- **Internal Node** - Any node, excluding the root, with at least one child is an **internal node**.
- **Siblings** - Nodes that share the same parent are called **siblings**.
- **Height of a Node** - The **height** of a node is the number of edges on the longest path from the node to a leaf.
- **Depth of a Node** - The **depth** of a node is the number of edges from the tree's root to the node.
- **Level** - In a tree, the root is at **level** 0. Its children are at level 1. The children of the children are at level 2, etc.
- **Degree** - The **degree** of a node is the total number of children (or branches) of that node.

Types of Trees

Several types of trees exist, each with its unique characteristics and applications. We will cover each of these trees in future assignments:

- **Binary Tree** - Each node has at most two children, referred to as the left child and the right child.
- **Binary Search Tree** - A binary tree where the left child is less than the parent, and the right child is greater than the parent.
- **AVL Tree** - A self-balancing binary search tree.
- **B-Tree** - A balanced tree used in databases to store large amounts of data.

Tree Properties

Understanding the properties of trees is essential for analyzing and implementing them efficiently. Here are a few critical properties of trees:

- **The Maximum Number of Nodes on Level `l`**
  $[Maximum\ Nodes = 2^{(l-1)}]$
- **The Maximum Number of Nodes in a Tree of Height `h`**
  $[Maximum\ Nodes = 2^h - 1]$
- **For a Binary Tree with `L` Leaves:**
  $[Internal\ Nodes = L - 1]$
- **Height of a Binary Tree with `n` Nodes:**
  $[Minimum\ Height = \lceil \log_2(n + 1) \rceil]$

Representing a Tree

A tree can be represented using nodes where each node contains data and references to its children. Here, we will be demonstrating a simple tree representation using a user-defined class. Further, we will implement a method to print out the tree in a readable format. For simplicity, we will consider a binary tree in this example.

Creating the Node

Similar to a doubly linked list, we are going to create a node with two pointers. This node, however, does not point to the previous and next nodes in a linear data structure. Instead, the references point to the child nodes in a hierarchical structure. The `data` attribute holds an integer, while the `left` and `right` pointers are of type `TreeNode`. In the constructor, give the `data` attribute a value and set `left` and `right` to `nullptr`.

```
class TreeNode {
public:
    int data; // data of the node
    TreeNode* left; // reference to the left child
    TreeNode* right; // reference to the right child

    // Constructor to initialize the node with data and null
        children
    TreeNode(int data) {
        this->data = data;
        this->left = nullptr;
        this->right = nullptr;
    }
};
```

Building the Tree

Once the `TreeNode` class is defined, you can start building the tree by creating nodes and linking them together. Create the `BinaryTree` class that has a `root` attribute. In the constructor, assign `root` a new `TreeNode` instance.

```
class BinaryTree {
public:
    TreeNode* root; // root of the tree

    BinaryTree(int data) {
        root = new TreeNode(data); // initialize the root
    }

};
```

The logic behind inserting nodes and other operations depend on the type of tree being used. For now, we are not going to worry about common operations, though they will be covered in future assignments.

Printing the Tree

To properly print a tree, you need to traverse it. Traversal is an operation that will be covered in more detail in the next assignment. Instead, we are going to manually walk through a tree with exactly five nodes:

```
          1
        /   \
       2     3
      / \
     4   5
```

Create the `printTree` method. Start by printing the data stored in the root node. Then print the the data from the children of the root (`2` and `3`). Only the left child (`2`) has children, so the final step is to print them.

```
void printTree() {
    std::cout << root->data << " ";

    std::cout << root->left->data << " ";
    std::cout << root->right->data << " ";

    std::cout << root->left->left->data << " ";
    std::cout << root->left->right->data << std::endl;
}
```

Again, we will cover more sophisticated ways to traverse and print the values stored in a tree in a later assignment.

▼ Code
Your code should look like this:

```
class TreeNode {
public:
    int data; // data of the node
    TreeNode* left; // reference to the left child
    TreeNode* right; // reference to the right child

    // Constructor to initialize the node with data and null
        children
    TreeNode(int data) {
        this->data = data;
        this->left = nullptr;
        this->right = nullptr;
    }
};

class BinaryTree {
public:
    TreeNode* root; // root of the tree

    BinaryTree(int data) {
        root = new TreeNode(data); // initialize the root
    }

    void printTree() {
        std::cout << root->data << " ";

        std::cout << root->left->data << " ";
        std::cout << root->right->data << " ";

        std::cout << root->left->left->data << " ";
        std::cout << root->left->right->data << std::endl;
    }
};
```

Create the main method and instantiate a `BinaryTree` object. Give the tree a `root` with the value of `1`, two children with the values `2` and `3`. Then node `2` should have the children `4` and `5`. Finally, print the tree.

```
int main() {
    BinaryTree tree(1); // root
    tree.root->left = new TreeNode(2); // left child of root
    tree.root->right = new TreeNode(3); // right child of root
    tree.root->left->left = new TreeNode(4); // left child of 2
    tree.root->left->right = new TreeNode(5); // right child of
        2

    tree.printTree(); // This will print: 1 2 3 4 5

    return 0;
}
```

This page provides a simple representation and a way to print out the tree. The representation can be expanded to include additional methods for inserting, deleting, searching nodes, and other tree operations as per the requirements of your specific application.

Formative Assessment 1

Formative Assessment 2